

Data Structures

Sheesh...and all this time I could have just let *Java* put things in alphabetical order? Third grade really sucks. We never learn anything useful...

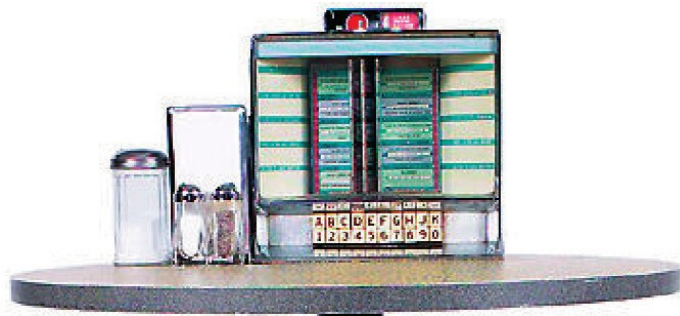


We'll use this "mock" class to test our code.

We'll make this method static, because this class doesn't have any instance fields and doesn't need any.

```
class MockSongs {  
    public static List<String> getSongStrings() {  
        List<String> songs = new ArrayList<>();  
        songs.add("somersault");  
        songs.add("cassidy");  
        songs.add("$10");  
        songs.add("havana");  
        songs.add("Cassidy");  
        songs.add("50 Ways");  
        return songs;  
    }  
}
```

This will be our list of six song titles to work with.



Because ArrayList IS-A List, we can create an ArrayList, store it in a List, and return List from the method.

In the real world you'll often see Java code that returns the interface type (List) and hides the implementation type (ArrayList).

```
import java.util.*;
```

```
public class Jukebox1 {
```

```
    public static void main(String[] args) {  
        new Jukebox1().go();  
    }
```

```
    public void go() {
```

```
        List<String> songList = MockSongs.getSongStrings();
```

```
        System.out.println(songList);  
    }
```

```
}
```

We'll store the song titles in a List of Strings.

Then print the contents of the songList.

File Edit Window Help Dance

```
%java Jukebox1
```

```
[somersault, cassydy, $10, havana,  
Cassidy, 50 Ways]
```

This is definitely NOT alphabetical!

```
public void go() {  
    List<String> songList = MockSongs.getSongStrings();  
    System.out.println(songList);  
    Collections.sort(songList);  Sort our song titles using  
    "natural ordering."  
    System.out.println(songList);  
}  
}
```

File Edit Window Help Dance

```
%java Jukebox1
```

```
[somersault, cassidy, $10, havana, Cassidy, 50 Ways]
```

```
[$10, 50 Ways, Cassidy, cassidy, havana, somersault]
```

Our songs unsorted,
in the order they
were added.



Our songs sorted.
Notice how the special
characters, numbers,
and uppercase letters
got sorted.



```
class SongV2 {  
    private String title;  
    private String artist;  
    private int bpm;
```

Three instance variables for the three song attributes in the file.

The variables are all set in the constructor whenever a new Song is created.

```
    SongV2(String title, String artist, int bpm) {  
        this.title = title;  
        this.artist = artist;  
        this.bpm = bpm;
```

```
    }  
    public String getTitle() {  
        return title;  
    }
```

```
    public String getArtist() {  
        return artist;  
    }
```

```
    public int getBpm() {  
        return bpm;  
    }
```

```
    public String toString() {  
        return title;  
    }
```

The getter methods for the three attributes.

We override toString(), because when you do a System.out.println(aSongObject), we want to see the title. When you do a System.out.println(aListOfSongs), it calls the toString() method of EACH element in the list.

```
}
```

```
class MockSongs {  
    public static List<String> getSongStrings() { ... }  
  
    public static List<SongV2> getSongsV2() {  
        List<SongV2> songs = new ArrayList<>();  
        songs.add(new SongV2("somersault", "zero 7", 147));  
        songs.add(new SongV2("cassidy", "grateful dead", 158));  
        songs.add(new SongV2("$10", "hitchhiker", 140));  
  
        songs.add(new SongV2("havana", "cabello", 105));  
        songs.add(new SongV2("Cassidy", "grateful dead", 158));  
        songs.add(new SongV2("50 ways", "simon", 102));  
        return songs;  
    }  
}
```

We made a new method in the MockSongs class to mock the new song data.


```
import java.util.*;
```

```
public class Jukebox2 {  
    public static void main(String[] args) {  
        new Jukebox2().go();  
    }
```

Change to a List of SongV2
objects instead of Strings.

```
    public void go() {
```

```
        List<SongV2> songList = MockSongs.getSongsV2();
```

```
        System.out.println(songList);
```

```
        Collections.sort(songList);
```

```
        System.out.println(songList);
```

```
    }
```

```
}
```

Call the mock class to load
song data into our List of
songs.

And once again, call the
sort method to sort the
songs.

File Edit Window Help Bummer

```
%javac Jukebox2.java
```

```
Jukebox2.java:13: error: no suitable method found for  
sort(List<SongV2>)
```

```
    Collections.sort(songList);
```

^

```
...
```

```
1 error
```

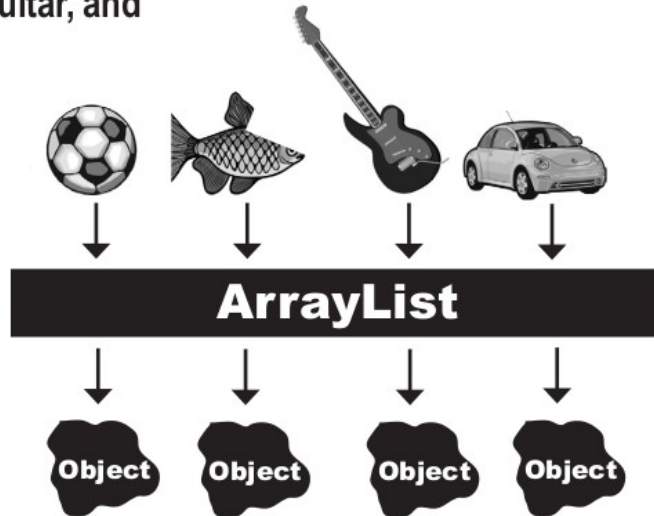
The sort() method declaration

```
static <T extends Comparable<? super T>> void sort(List<T> list)
```

Sorts the specified list into ascending order, according to the **natural ordering** of its elements.

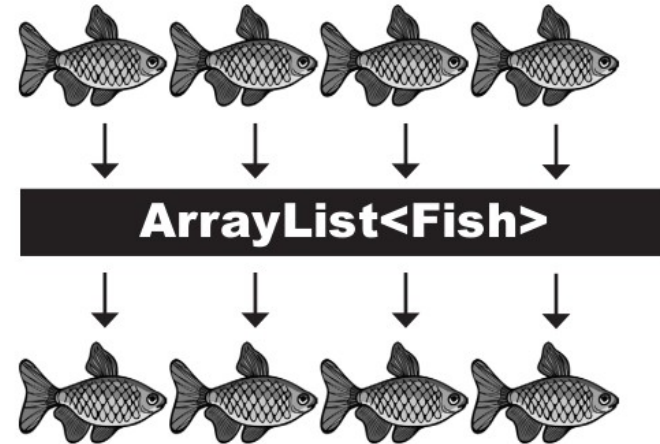
WITHOUT generics

Objects go IN as a reference to SoccerBall, Fish, Guitar, and Car objects



WITH generics

Objects go IN as a reference to only Fish objects



And come out as a reference of type Fish

The "E" is a placeholder for the REAL type you use when you declare and create an ArrayList.

ArrayList is a subclass of AbstractList, so whatever type you specify for the ArrayList is automatically used for the type of the AbstractList.

```
public class ArrayList<E> extends AbstractList<E> implements List<E> ... {
```

```
    public boolean add(E o)
```

Here's the important part! Whatever "E" is determines what kind of things you're allowed to add to the ArrayList.

The type (the value of <E>) becomes the type of the List interface as well.

```
    // more code
```

```
}
```

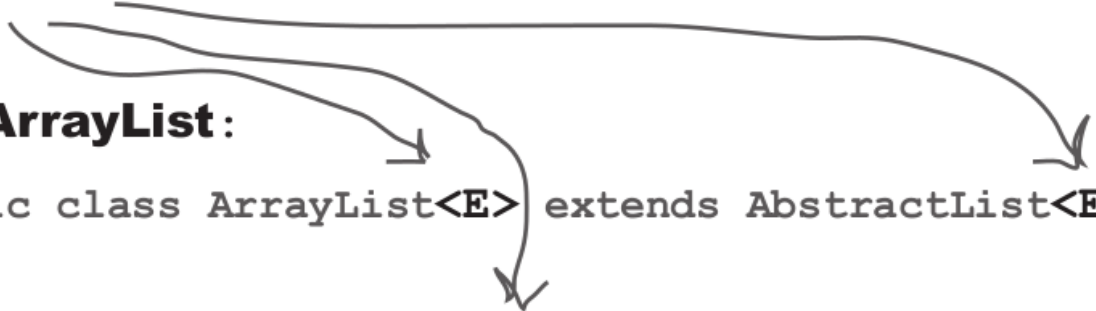
Think of "E" as a stand-in for "the type of element you want this collection to hold and return." (E is for Element.)

THIS code:

```
List<String> thisList = new ArrayList<>
```

Means ArrayList:

```
public class ArrayList<E> extends AbstractList<E> ... {  
  
    public boolean add(E o)  
    // more code  
}
```



Is treated by the compiler as:

```
public class ArrayList<String> extends AbstractList<String>... {  
  
    public boolean add(String o)  
    // more code  
}
```

Using generic METHODS

- ① Using a type parameter defined in the class declaration

```
public class ArrayList<E> extends AbstractList<E> ... {  
    public boolean add(E o)
```

You can use the "E" here *ONLY* because it's already been defined as part of the class.

- ② Using a type parameter that was NOT defined in the class declaration

```
public <T extends Animal> void takeThing(ArrayList<T> list)
```

Here we can use <T> because we declared "T" at the start of the method declaration.

```
static <T extends Comparable<? super T>> void sort(List<T> list)
```

Sorts the specified list into ascending order, according to the **natural ordering** of its elements.

The sort() method can take only lists of Comparable objects.

Song is NOT a subtype of Comparable, so you cannot sort() the list of Songs.

At least not yet...

```
public static <T extends Comparable<? super T>> void sort(List<T> list)
```

This says "Whatever 'T' is must be of type Comparable."

(Ignore this part for now. But if you can't, it just means that the type parameter for Comparable must be of type T or one of T's supertypes.)

You can pass in only a List (or subtype of list, like ArrayList) that uses a parameterized type that "extends Comparable."



Um...I just checked the docs for String, and String doesn't EXTEND Comparable—it IMPLEMENTS it. **Comparable** is an interface. So it's nonsense to say `<T extends Comparable>`.

```
public final class String  
    implements java.io.Serializable, Comparable<String>, CharSequence {
```

In generics, “extends” means “extends or implements”

In generics, the keyword
“extends” really means “IS-A”
and works for BOTH classes
and interfaces.

Comparable is an interface, so this
REALLY reads, “T must be a type that
implements the Comparable interface.”



```
public static <T extends Comparable<? super T>> void sort(List<T> list)
```

It doesn't matter whether the thing on the right is
a class or interface...you still say “extends.”

java.lang.Comparable

```
public interface Comparable<T> {  
    int compareTo(T o);  
}
```

Returns:

a negative integer if this object is less than the specified object;
a zero if they're equal;
a positive integer if this object is greater than the specified object.

Usually these match...we're specifying the type that the implementing class can be compared against.

This means that SongV3 objects can be compared to other SongV3 objects, for the purpose of sorting.

```
class SongV3 implements Comparable<SongV3> {
```

```
    private String title;  
    private String artist;  
    private int bpm;
```

```
    public int compareTo(SongV3 s) {  
        return title.compareTo(s.getTitle());  
    }
```

The sort() method sends a Song to compareTo() to see how that Song compares to the Song on which the method was invoked.

Simple! We just pass the work on to the title String objects, since we know Strings have a compareTo() method.

```
    SongV3(String title, String artist, int bpm) {  
        this.title = title;  
        this.artist = artist;  
        this.bpm = bpm;  
    }
```

That's not good enough.
Sometimes I want it to sort
by artist instead of title.



Excerpts from the API

java.util.Collections

sort(List): Sorts the specified list into ascending order, according to the **natural ordering** of its elements.

sort(List, **Comparator**): Sorts the specified list according to the order induced by the specified **Comparator**.

java.util.List

sort(**Comparator**): Sorts this list according to the order induced by the specified **Comparator**.

The sort() method on Collections is overloaded to take something called a Comparator.

There's also a sort() on List, which takes a Comparator.

java.util.Comparator

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
}
```

If you pass a `Comparator` to the `sort()` method, the sort order is determined by the `Comparator`.

If you don't pass a `Comparator` and the element is `Comparable`, the sort order is determined by the element's `compareTo()` method.

```
class ArtistCompare implements Comparator<SongV3> {  
    public int compare(SongV3 one, SongV3 two) {  
        return one.getArtist().compareTo(two.getArtist());  
    }  
}
```

This is a String (the artist)

We're letting the `String` variables (for artist) do the actual comparison, since `Strings` already know how to alphabetize themselves.

```
import java.util.*;
```

```
public class Jukebox4 {  
    public static void main(String[] args) {  
        new Jukebox4().go();  
    }
```

```
    public void go() {  
        List<SongV3> songList = MockSongs.getSongsV3();  
        System.out.println(songList);
```

```
        Collections.sort(songList);  
        System.out.println(songList);
```

```
        ArtistCompare artistCompare = new ArtistCompare();  
        songList.sort(artistCompare);  
        System.out.println(songList);
```

```
    }
```

```
}
```

Make an instance of the
Comparator class.



Invoke sort() on our list, passing
it a reference to the new custom
Comparator object.



The compiler cares not a whit what you call the class.

Doh! The compiler can infer this from the sort() does.

```
class TitleCompare implements Comparator<Song> {
```

Yup, the compiler knows what this method should look like.

```
    public int compare(Song one, Song two) {
```

```
        return one.getTitle().compareTo(two.getTitle());
```

The compiler can figure out that the two objects have to be Song objects since songList is a List of Song objects.

Or, we could use a lambda...

```
songList.sort((one, two) -> one.getTitle().compareTo(two.getTitle()));
```

These are Song one and Song two, the parameters to the compare method.

Some interfaces have only ONE method to implement

With interfaces like Comparator, we only have to implement a *single abstract method*, SAM for short. These interfaces are so important that they have several special names:

SAM Interfaces a.k.a. Functional Interfaces

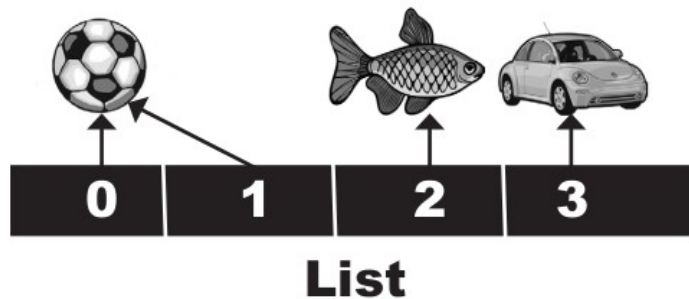
If an interface has only one method that needs to be implemented, that interface can be implemented as a *lambda expression*. You don't need to create a whole class to implement the interface; the compiler knows what the class and method would look like. What the compiler *doesn't* know is the logic that goes *inside* that method.

► **LIST** - when *sequence* matters

Collections that know about *index position*.

Lists know where something is in the list. You can have more than one element referencing the same object.

Duplicates OK.

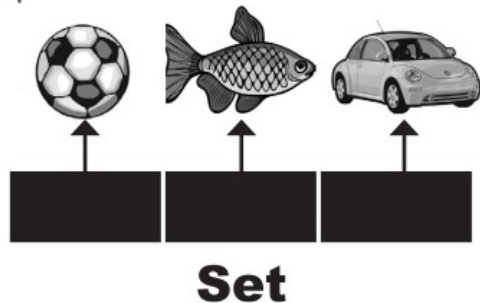


► **SET** - when *uniqueness* matters

Collections that *do not allow duplicates*.

Sets know whether something is already in the collection. You can never have more than one element referencing the same object (or more than one element referencing two objects that are considered equal—we'll look at what object equality means in a moment).

NO duplicates.

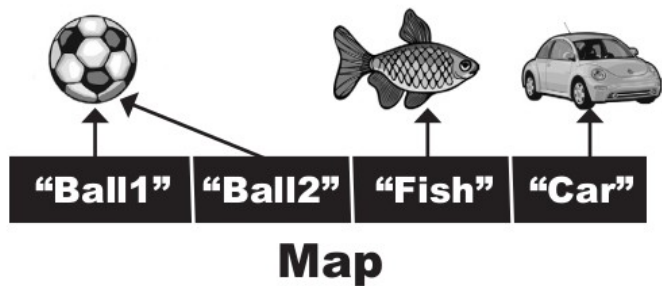


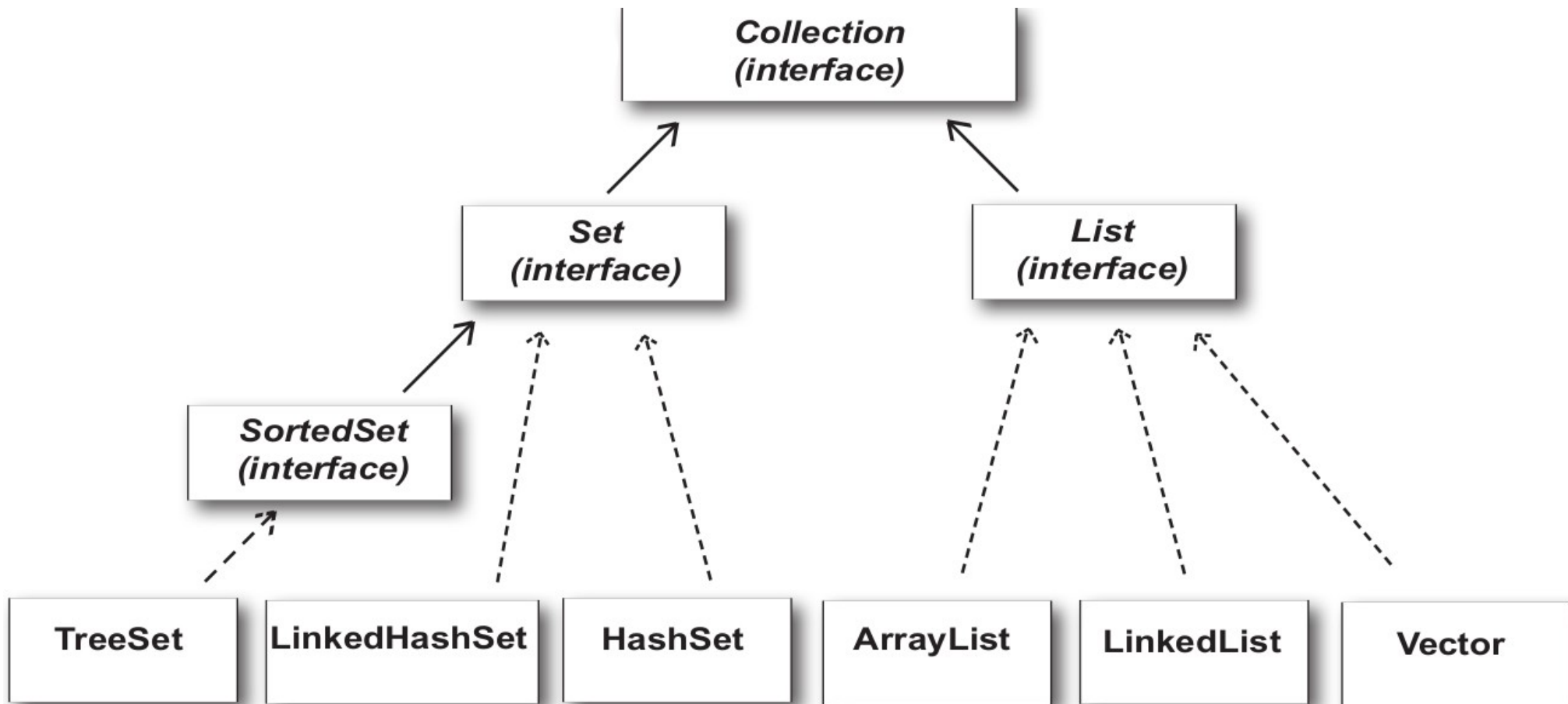
► **MAP** - when *finding something by key* matters

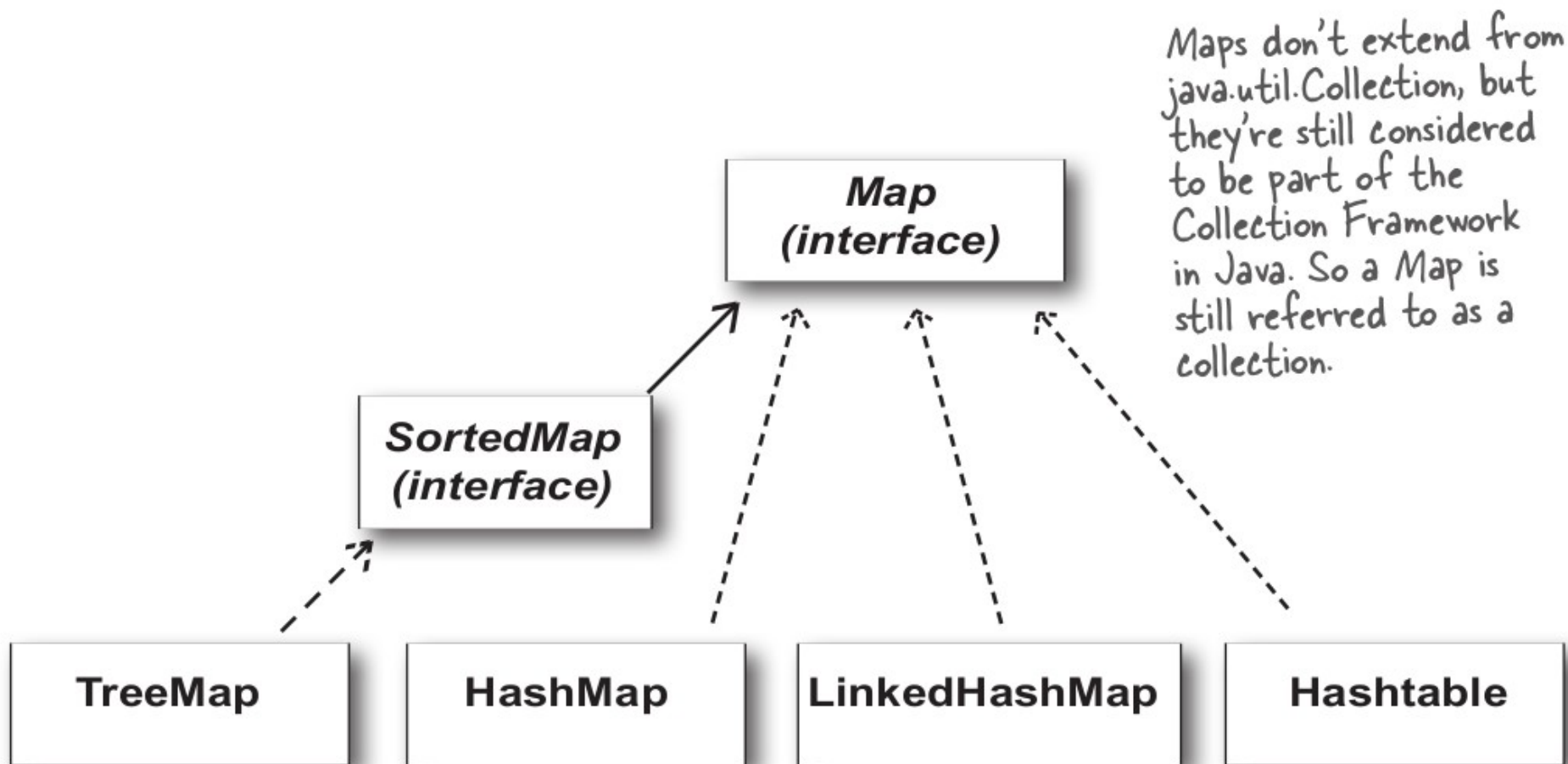
Collections that use *key-value pairs*.

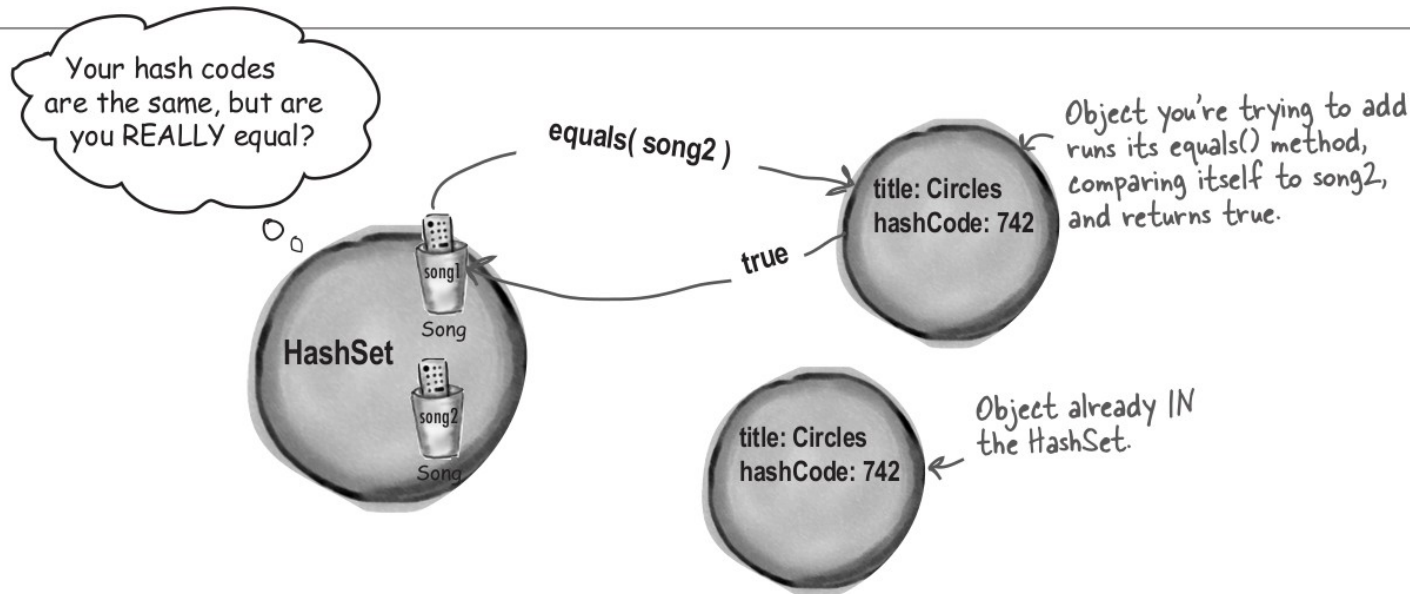
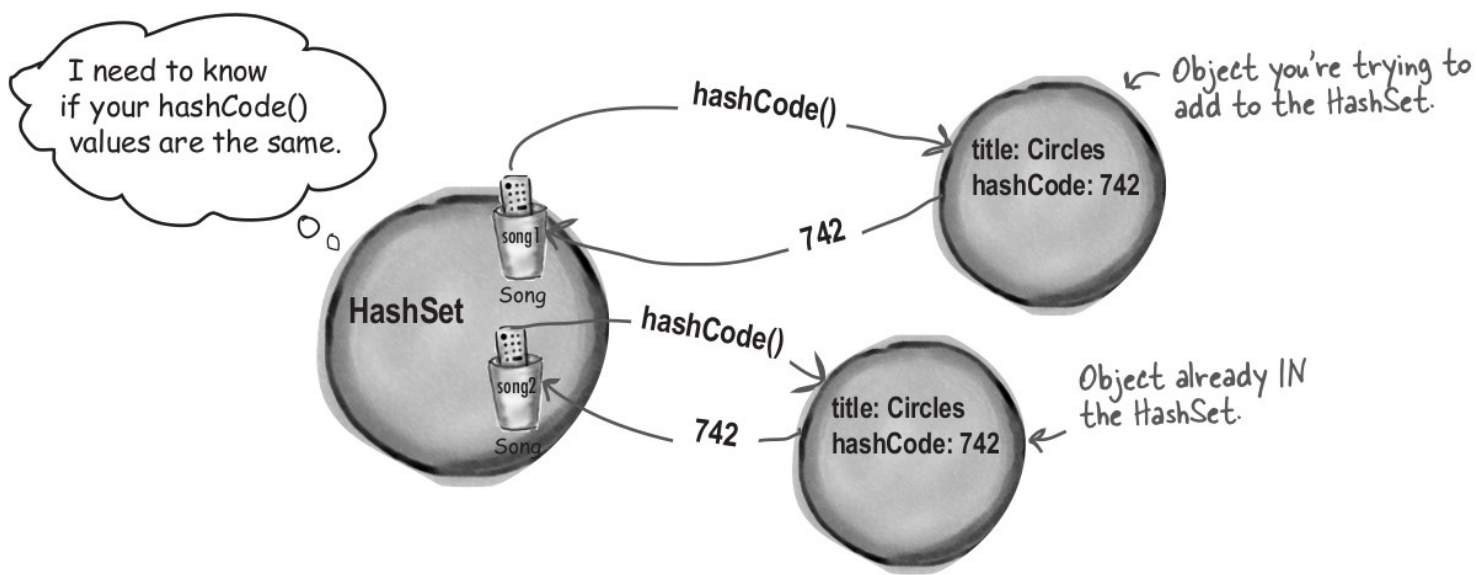
Maps know the value associated with a given key. You can have two keys that reference the same value, but you cannot have duplicate keys. A key can be any object.

Duplicate values OK, but NO duplicate keys.









The Song class with overridden hashCode() and equals()

```
class SongV4 implements Comparable<SongV4> {  
    private String title;  
    private String artist;  
    private int bpm;
```

```
    public boolean equals(Object aSong) {  
        SongV4 other = (SongV4) aSong;  
        return title.equals(other.getTitle());  
    }
```

```
    public int hashCode() {  
        return title.hashCode();  
    }
```

The HashSet (or anyone else calling this method) sends it another Song.

The GREAT news is that title is a String, and Strings have an overridden equals() method. So all we have to do is ask one title if it's equal to the other song's title.

Same deal here...the String class has an overridden hashCode() method, so you can just return the result of calling hashCode() on the title. Notice how hashCode() and equals() are using the SAME instance variable (title).

```
public class Jukebox10 {  
    public static void main(String[] args) {  
        new Jukebox10().go();  
    }  
  
    public void go() {  
        List<SongV4> songList = MockMoreSongs.getSongsV4();  
        System.out.println(songList);  
  
        songList.sort((one, two) -> one.getTitle().compareTo(two.getTitle()));  
        System.out.println(songList);  
  
        Set<SongV4> songSet = new TreeSet<>(songList);  
        System.out.println(songSet);  
    }  
}
```

Create a TreeSet instead of HashSet. The TreeSet will use SongV4's compareTo() method to sort the items in songList.

```
Set<SongV4> songSet = new TreeSet<>((o1, o2) -> o1.getBpm() - o2.getBpm());  
songSet.addAll(songList);
```

Yep, another lambda for sorting. This one sorts by BPM. Remember, this lambda implements Comparator.



```
class Book {  
    private String title;  
    public Book(String t) {  
        title = t;  
    }  
}
```

- 1) What is the result when you compile this code?
- 2) If it compiles, what is the result when you run the TestTree class?
- 3) If there is a problem (either compile-time or runtime) with this code, how would you fix it?

```
import java.util.*;  
  
public class TestTree {  
    public static void main(String[] args) {  
        new TestTree().go();  
    }  
  
    public void go() {  
        Book b1 = new Book("How Cats Work");  
        Book b2 = new Book("Remix your Body");  
        Book b3 = new Book("Finding Emo");  
  
        Set<Book> tree = new TreeSet<>();  
        tree.add(b1);  
        tree.add(b2);  
        tree.add(b3);  
        System.out.println(tree);  
    }  
}  
  
class Book {  
    private String title;  
    public Book(String t) {  
        title = t;  
    }  
}
```

```

class Book implements Comparable<Book> {
    private String title;
    public Book(String t) {
        title = t;
    }

    public int compareTo(Book other) {
        return title.compareTo(other.title);
    }
}

```

OR

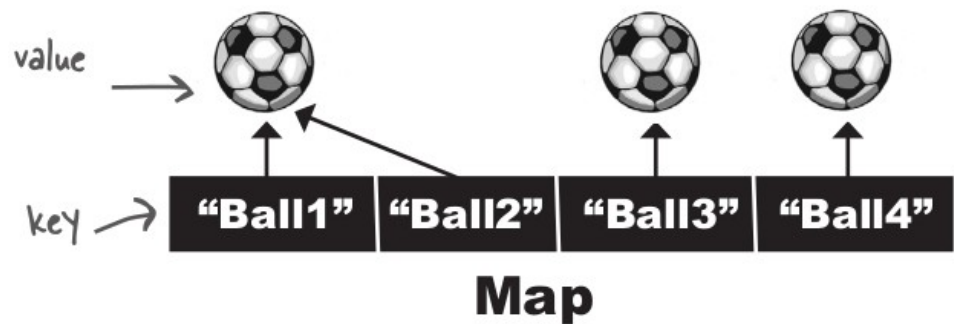
```

class BookCompare implements Comparator<Book> {
    public int compare(Book one, Book two) {
        return one.title.compareTo(two.title);
    }
}

public class TestTreeComparator {
    public void go() {
        Book b1 = new Book("How Cats Work");
        Book b2 = new Book("Remix your Body");
        Book b3 = new Book("Finding Emo");
        BookCompare bookCompare = new BookCompare();
        Set<Book> tree = new TreeSet<>(bookCompare);
        tree.add(b1);
        tree.add(b2);
        tree.add(b3);
        System.out.println(tree);
    }
}

```

You could use a lambda instead of declaring a new Comparator class.



Each element in a Map is actually TWO objects—a key and a value. You can have duplicate values, but NOT duplicate keys.

```
public class TestMap {  
    public static void main(String[] args) {  
        Map<String, Integer> scores = new HashMap<>();
```

HashMap needs TWO type parameters—one for the key and one for the value.

```
        scores.put("Kathy", 42);  
        scores.put("Bert", 343);  
        scores.put("Skyler", 420);
```

Use put() instead of add(), and now of course it takes two arguments (key, value).

```
        System.out.println(scores);  
        System.out.println(scores.get("Bert"));
```

The get() method takes a key and returns the value (in this case, an Integer).

```
    }  
}
```

I keep seeing the same code popping up again and again for creating collections. There must be something we can do to make this easier for people.

Good point! Let me just whip up some Factory methods.



Convenience Factory Methods for Collections

Convenience Factory Methods for Collections allow you to easily create a List, Set, or Map that's been prefilled with known data. There are a couple of things to understand about using them:

- ① **The resulting collections cannot be changed.** You can't add to them or alter the values; in fact, you can't even do the sorting that we've seen in this chapter.
- ② **The resulting collections are not the standard Collections we've seen.** These are not ArrayList, HashSet, HashMap, etc. You can rely on them to behave according to their interface: a List will always preserve the order in which the elements were placed; a Set will never have duplicates. But you can't rely on them being a specific implementation of List, Set, or Map.

► Creating a List: List.of()

[illegible]

► Creating a Set: Set.of()

```
Set<Book> books = Set.of(new Book("How Cats Work"),
                           new Book("Remix your Body"),
                           new Book("Finding Emo"));
```

► Creating a Map: Map.of(), Map.ofEntries()

[illegible]


```
public void go() {  
    List<Animal> animals = List.of(new Dog(), new Cat(), new Dog());  
    takeAnimals(animals);
```

← We know this line worked fine.

```
    List<Dog> dogs = List.of(new Dog(), new Dog());  
    takeAnimals(dogs);  
}
```

← Will this work now that we changed
from an array to a List?

Make a Dog List and
put a couple dogs in.

```
public void takeAnimals(List<Animal> animals) {  
    for (Animal a : animals) {  
        a.eat();  
    }  
}
```

File Edit Window Help CatsAreSmarter

```
%javac TestGenerics2.java
```

```
TestGenerics2.java:20: error: incompatible types:  
List<Dog> cannot be converted to List<Animal>  
    takeAnimals(dogs);  
                  ^
```

```
1 error
```



```
public void takeAnimals(List<Animal> animals) {  
    animals.add(new Cat()); ← Yikes!! We just stuck a Cat in what  
}
```

might be a Dogs-only List.

If you declare a method to take
List<Animal>, it can take **ONLY** a
List<Animal>, not **List<Dog>** or **List<Cat>**.

```
public void takeAnimals(List<? extends Animal> animals) {  
    for (Animal a : animals) {  
        a.eat();  
    }  
}
```

Remember, the keyword "extends" here
means either extends OR implements.

Using the method's generic type parameter

```
public <T extends Animal> List<T> takeAnimals(List<T> list) { }
```

```
List<Dog> dogs = List.of(new Dog(), new Dog());
```

```
List<Dog> vaccinatedDogs = takeAnimals(dogs);
```

The List we get back from the takeAnimals method is always the same type as the list we pass in.

```
List<Animal> animals = List.of(new Dog(), new Cat());
```

```
List<Animal> vaccinatedAnimals = takeAnimals(animals);
```

```
public void go() {
```

```
    List<Dog> dogs = List.of(new Dog(), new Dog());
```

```
    List<? extends Animal> vaccinatedSomethings = takeAnimals(dogs);
```

```
}
```

```
public List<? extends Animal> takeAnimals(List<? extends Animal> animals) { }
```

Using the wildcard (“?” extends”) is fine when you don’t care much about the generic type, you just want to allow all subtypes of some type.

Using a type parameter (“T”) is more helpful when you want to do more with the type itself, for example in the method’s return.